# Course Title:

BIT302 Software Engineering

# Course Contents

**Unit 4: System Modeling**

Context Models,
Interaction Models,
Structural Models,
Behavioral Models,
Model Driven Architecture

# Course Contents

## Unit 4: System Modeling


5.1 Context Models,

5.2 Interaction Models,
- 5.2.1 Use case modeling
- 5.2.2 Sequence diagrams

5.3 Structural Models,
- 5.3.1 Class diagrams
- 5.3.2 Generalization
- 5.3.3 Aggregation

5.4 Behavioral Models,
- 5.4.1 Data-driven modeling
- 5.4.2 Event-driven modeling
- 5.4.3 Model-driven engineering

5.5 Model Driven Architecture


# Unit 4: System Modeling

## Objectives:

- Introduce system models that may be developed as part of requirements engineering and system design processes.
- Understand how graphical models can be used to represent software systems and why several types of model are needed to fully represent a system;
- Understand the fundamental system modeling perspectives of context, interaction, structure, and behavior;
- Understand the principal diagram types in the Unified Modeling Language (UML) and how these diagrams may be used in system modeling;
- Introduce model-driven engineering, where an executable system is automatically generated from structural and behavioral models.

# Unit 4: System Modeling

## System Modeling

- **System modeling** is the process of developing abstract models of a system, with each model presenting a different view or perspective of that system.
- System modeling to mean representing a system using some kind of graphical notation, which is now almost always based on notations in the Unified Modeling Language (UML).
- System modelling helps the analyst to understand the functionality of the system and models are used to communicate with customers.

# Unit 4: System Modeling

## System Modeling (Existing and planned system models):

Models of both new and existing system are used during **requirements engineering**. Models of the **existing systems** help clarify what the existing system does and can be used as a basis for discussing its strengths and weaknesses. These then lead to requirements for the new system. Models of the **new system** are used during requirements engineering to help explain the proposed requirements to other system stakeholders. Engineers use these models to discuss design proposals and to document the system for implementation.

- Models of the existing system can help clarify what the existing system does and can be used as a basis for discussing its strengths and weaknesses. These then can lead to requirements for the new system.
- Models of the new system can help explain the proposed system to stakeholders. Engineers use these models to discuss design proposals and to document the system for implementation.
- In a model-driven engineering process, it is possible to generate a complete or partial system implementation from the system model.

## Unit 4: System Modeling

**System perspectives:**

We may develop different models to represent the system from different perspectives :

- An external perspective, where you model the context or environment of the system.
- An interaction perspective, where you model the interactions between a system and its environment, or between the components of a system.
- A structural perspective, where you model the organization of a system or the structure of the data that is processed by the system.
- A behavioral perspective, where you model the dynamic behavior of the system and how it responds to events.

## Unit 4: System Modeling

**System perspectives:**

- Functional perspective, where you model functionality of the system.
- Structural perspective, where you model the organization of a system or the structure of the data that is processed by the system.
- A behavioral perspective (interaction perspective) where you model the dynamic behavior of the system (interactions between a system) and how it responds to events.

# Unit 4: System Modeling

**UML diagram types:** Unified Modeling Language (UML) has become a standard language for object-oriented modeling. The UML has 13 diagram types to supports the creation of many different types of system model. However, most users of the UML thought that five diagram types could represent the essentials of a system. Five types of UML diagrams that are the most useful for system modeling:

- Activity diagrams, which show the activities involved in a process or in data processing.
- Use case diagrams, which show the interactions between a system and its environment.
- Sequence diagrams, which show interactions between actors and the system and between system components.
- Class diagrams, which show the object classes in the system and the associations between these classes.
- State diagrams, which show how the system reacts to internal and external events.

# Unit 4: System Modeling

**Use of graphical models:**

- As a means of facilitating discussion about an existing or proposed system
  - Incomplete and incorrect models are OK as their role is to support discussion.
- As a way of documenting an existing system
  - Models should be an accurate representation of the system but need not be complete.
- As a detailed system description that can be used to generate a system implementation
  - Models have to be both correct and complete.

# Unit 4: System Modeling

**Context models:**

- At an early stage in the specification of a system, you should decide on the system boundaries, that is, on what is and is not part of the system being developed. This involves working with system stakeholders to decide what functionality should be included in the system and what processing and operations should be carried out in the system's operational environment.
- We may decide that automated support for some business processes should be implemented in the software being developed but that other processes should be manual or supported by different systems.
- We should look at possible overlaps in functionality with existing systems and decide where new functionality should be implemented.
- These decisions should be made early in the process to limit the system costs and the time needed for understanding the system requirements and design.

# Unit 4: System Modeling

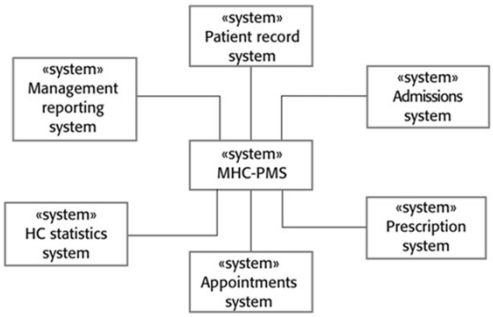
**Context models:**

- Common techniques for creating context models include context diagrams, data flow diagrams, and use case diagrams. These visual representations help stakeholders visualize the system's interactions with its environment and provide a foundation for further system analysis, design, and development.
- Normally, producing a simple architectural model is the first step in this activity.
- Figure in next slide is a **context model** that shows the Mentcare patient information system and the other systems in its environment.

# Unit 4: System Modeling

**Context models:**

Context model that shows the Mentcare system and the other systems in its environment. Mentcare system is connected to an appointments system and a more general patient record system with which it shares data. The system is also connected to systems for management reporting and hospital admissions, and a statistics system that collects information for research. Finally, it makes use of a prescription system to generate prescriptions for patients' medication.



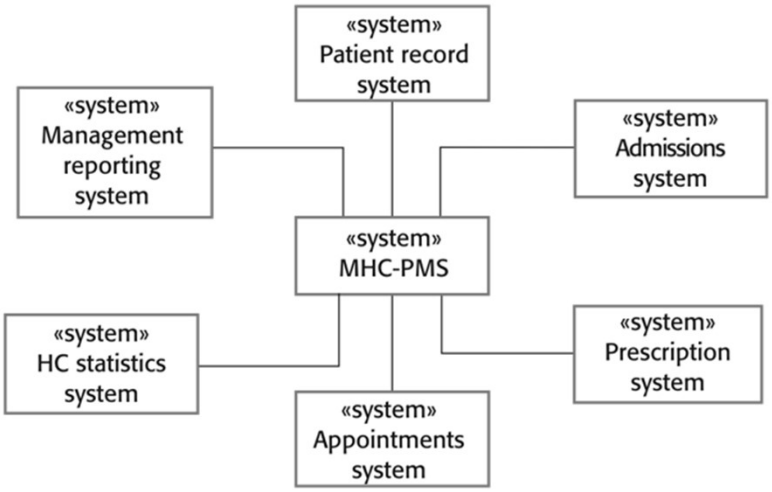

# Unit 4: System Modeling

**Context models:**

- Context models are used to illustrate the operational context of a system - they show what lies outside the system boundaries.
- Social and organizational concerns may affect the decision on where to position system boundaries.
- Architectural models show the system and its relationship with other systems.
- System boundaries are established to define what is inside and what is outside the system. They show other systems that are used or depend on the system being developed. The position of the system boundary has a profound effect on the system requirements. Defining a system boundary is a political judgment since there may be pressures to develop system boundaries that increase/decrease the influence or workload of different parts of an organization.
- Context models simply show the other systems in the environment, not how the system being developed is used in that environment.

# Unit 4: System Modeling

**Example: The context model of the MHC-PMS :**



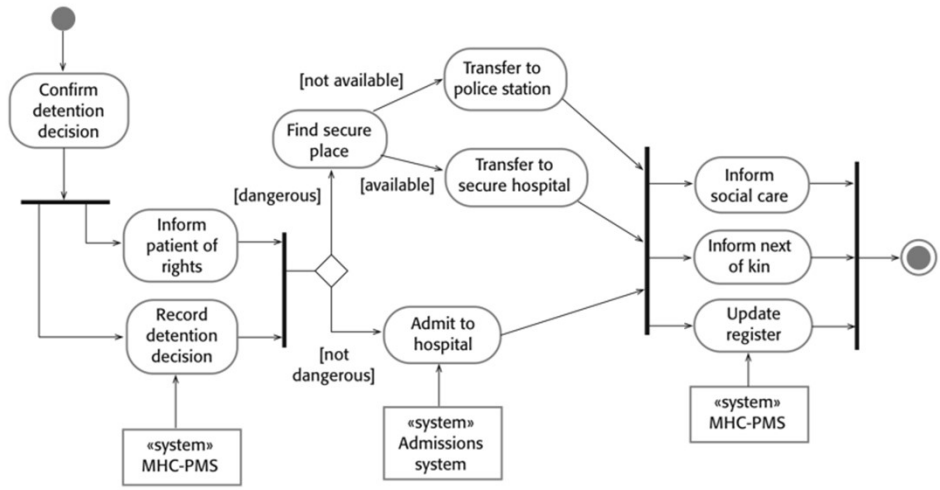

# Course Contents

**Process perspective:**

- Context models simply show the other systems in the environment, not how the system being developed is used in that environment.
- Simple context models are used along with other models, such as business process models which describe human and automated processes in which particular software systems are used.
- Process models reveal how the system being developed is used in broader business processes. UML activity diagrams may be used to show the business processes in which systems are used.
- Figure in next slide is a UML activity diagram that shows where the Mentcare system is used in an important mental health care process - involuntary detention.

# Course Contents

**Process model of involuntary detention (UML activity diagrams):**
MHC-PMS (Mental Healthcare Patient Management System)




# Course Contents

**UML activity diagrams:**

UML activity diagrams show the activities in a process and the flow of control from one activity to another.

- The start of a process is indicated by a filled circle, the end by a filled circle inside another circle.
- Rectangles with round corners represent activities, that is, the specific sub-processes that must be carried out. You may include objects in activity charts.
- The systems that are used to support different sub-processes and represent by UML stereotype feature where the type of entity in the box is shown.
- Arrows represent the flow of work from one activity to another, and a solid bar indicates activity coordination. When the flow from more than one activity leads to a solid bar, then all of these activities must be complete before progress is possible. When the flow from a solid bar leads to a number of activities, these may be executed in parallel.
- Arrows may be annotated with guards (in square brackets) that specify when that flow is followed. In Figure, can see guards showing the flows for patients who are dangerous and not dangerous to society.

# UML activity diagrams

**Process model of involuntary detention (UML activity diagrams):**
MHC-PMS (Mental Healthcare Patient Management System)

| Activities | Symbol | Example |
|---|---|---|
| Start of a process | filled circle | |
| End of a process | filled circle inside another circle | |
| Activities | Rectangles with round corners | Transfer to police station |
| Systems | Rectangular with stereotype feature where the type of entity in the box is shown | «system» MHC-PMS |
| Flow of work | Arrow | |
| Activity coordination | solid bar | |
| Condition of Flow | Arrows may be annotated with guards (in square brackets [ ]) | [dangerous] |

## Elements of an Activity Diagram:

The syntax of an activity diagram.

| | |
|---|---|
| **An action:**<br>■ Is a simple, nondecomposable piece of behavior.<br>■ Is labeled by its name. | Action |
| **An activity:**<br>■ Is used to represent a set of actions.<br>■ Is labeled by its name. | Activity |
| **An object node:**<br>■ Is used to represent an object that is connected to a set of object flows.<br>■ Is labeled by its class name. | Class Name |
| **A control flow:**<br>■ Shows the sequence of execution. | |
| **An object flow:**<br>■ Shows the flow of an object from one activity (or action) to another activity (or action). | |

## Elements of an Activity Diagram:

The syntax of an activity diagram.

| | |
|---|---|
| **An initial node:**<br>■ Portrays the beginning of a set of actions or activities. | |
| **A final-activity node:**<br>■ Is used to stop all control flows and object flows in an activity (or action). | |
| **A final-flow node:**<br>■ Is used to stop a specific control flow or object flow. | |
| **A decision node:**<br>■ Is used to represent a test condition to ensure that the control flow or object flow only goes down one path.<br>■ Is labeled with the decision criteria to continue down the specific path. | [Decision Criteria] [Decision Criteria] |
| **A merge node:**<br>■ Is used to bring back together different decision paths that were created using a decision node. | |

## Elements of an Activity Diagram:

The syntax of an activity diagram.

| | |
|---|---|
| **A fork node:**<br>Is used to split behavior into a set of parallel or concurrent flows of activities (or actions) | |
| **A join node:**<br>Is used to bring back together a set of parallel or concurrent flows of activities (or actions) | |

## Course Contents

**Interaction models:**

Different models from different perspectives :

- An external perspective, where you model the context or environment of the system.
- An interaction perspective, where you model the interactions between a system and its environment, or between the components of a system.
- A structural perspective, where you model the organization of a system or the structure of the data that is processed by the system.
- A behavioral perspective, where you model the dynamic behavior of the system and how it responds to events.

## Course Contents

**Interaction models:**

All systems involve interaction of some kind.

- This can be user interaction, which involves user inputs and outputs; interaction between the software being developed and other systems in its environment; or interaction between the components of a software system.
- **User interaction** modeling is important as it helps to identify user requirements. Modeling **system-to-system interaction** highlights the communication problems that may arise. Modeling **component interaction** helps us understand if a proposed system structure is likely to deliver the required system performance and dependability.

Two related approaches to interaction modeling are:

1. Use case modeling, which is mostly used to model interactions between a system and external agents (human users or other systems).
2. Sequence diagrams, which are used to model interactions between system components, although external agents may also be included.

# Course Contents

**Interaction models:**

Types of interactions that can be represented in a model:

- Modeling user interaction is important as it helps to identify user requirements
- Modeling system-to-system interaction highlights the communication problems that may arise.
- Modeling component interaction helps us understand if a proposed system structure is likely to deliver the required system performance and dependability. (Note: System software components are defined as the software system designed to operate the computer's hardware i.e. Device Driver, Operating system, Server sw etc.)

Use case models and sequence diagrams present interactions at different levels of detail and so may be used together. For example, the details of the interactions involved in a high-level use case may be documented in a sequence diagram. The UML also includes communication diagrams that can be used to model interactions.

# Course Contents

**Use case modeling:**

- Use cases were developed originally to support requirements elicitation and now are incorporated into the UML.
- Each use case represents a discrete task that involves external interaction with a system.
- Actors in a use case may be people, time, or other systems.

**Example: Transfer-data use case**

Notice that there are two actors in this use case—the operator who is transferring the data and the patient record system. Obviously, in a use case, messages pass in both directions. However, the arrows in Figure are used informally to indicate that the medical receptionist initiates the transaction and data is transferred to the patient record system.

# Course Contents

**Use case modeling:**

- Use case diagrams give a simple overview of an interaction, and need to add more detail for complete interaction description.
- The detail can either be a simple textual description, a structured description in a table, or a sequence diagram.
- We choose the most appropriate format depending on the use case and the level of detail that we think is required in the model.

# Course Contents

**Tabular description of the 'Transfer data' use-case:**

MHC-PMS: Transfer data

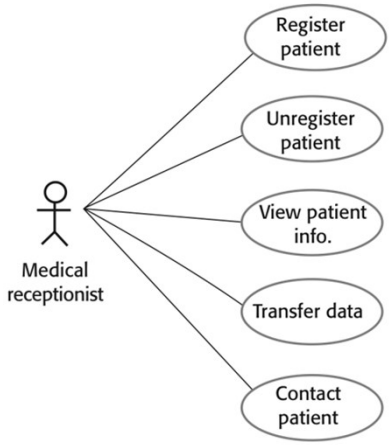
| | |
|---|---|
| Actors | Medical receptionist, patient records system (PRS) |
| Description | A receptionist may transfer data from the MHC-PMS to a general patient record database that is maintained by a health authority. The information transferred may either be updated personal information (address, phone number, etc.) or a summary of the patient's diagnosis and treatment. |
| Data | Patient's personal information, treatment summary |
| Stimulus | User command issued by medical receptionist |
| Response | Confirmation that PRS has been updated |
| Comments | The receptionist must have appropriate security permissions to access the patient information and the PRS. |

# Course Contents

**Use cases in the MHC-PMS involving the role 'Medical Receptionist':**

- Sometimes it is possible to include all possible interactions within a system in a single composite use case diagram.
- Composite use case diagrams show a number of different use cases.
- However, this may be impossible because of the number of use cases. In such cases, we may develop several diagrams, each of which shows related use cases.

For example, Figure shows all of the use cases in the Mentcare system in which the actor "Medical Receptionist" is involved. Each of these should be accompanied by a more detailed description.




# Use-Case Diagrams

## Elements of Use-Case Diagram

An analyst can employ use cases to better understand the functionality of the system at a very high level. Typically, because a use-case diagram provides a simple, straightforward way of communicating to the users exactly what the system will do, a use-case diagram is drawn when gathering and defining requirements for the system.

The use-case diagram can encourage the users to provide additional high-level requirements. A use-case diagram illustrates in a very simple way the main functions of the system and the different kinds of users that will interact with it.

# Use-Case Diagrams

## Elements of Use-Case Diagrams:

The elements of a use-case diagram include

1. actors,
2. use cases,
3. subject boundaries,

and a set of relationships among actors, actors and use cases, and use cases. These relationships consist of

4. association,
5. include,
6. extend,
7. generalization relationships.

# Use-Case Diagrams

## Elements of Use-Case Diagrams:

**An actor:**

- Is a person or system that derives benefit from and is external to the subject.
- Is depicted as either a stick figure (default) or, if a nonhuman actor is involved, a rectangle with <<actor>> in it (alternative).
- Is labeled with its role.
- Can be associated with other actors using a specialization/superclass association, denoted by an arrow with a hollow arrowhead.
- Is placed outside the subject boundary.

Actor/Role

<<actor>>
Actor/Role

**A use case:**

- Represents a major piece of system functionality.
- Can extend another use case.
- Can include another use case.
- Is placed inside the system boundary.
- Is labeled with a descriptive verb–noun phrase.

Use Case

**A subject boundary:**

- Includes the name of the subject inside or on top.
- Represents the scope of the subject, e.g., a system or an individual business process.

Subject

# Use-Case Diagrams

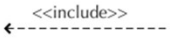
## Elements of Use-Case Diagrams:

**An association relationship:**

- Links an actor with the use case(s) with which it interacts.

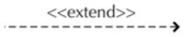
**An include relationship:**

- Represents the inclusion of the functionality of one use case within another.
- Has an arrow drawn from the base use case to the used use case.




**An extend relationship:**

- Represents the extension of the use case to include optional behavior.
- Has an arrow drawn from the extension use case to the base use case.




**A generalization relationship:**

- Represents a specialized use case to a more generalized one.
- Has an arrow drawn from the specialized use case to the base use case.

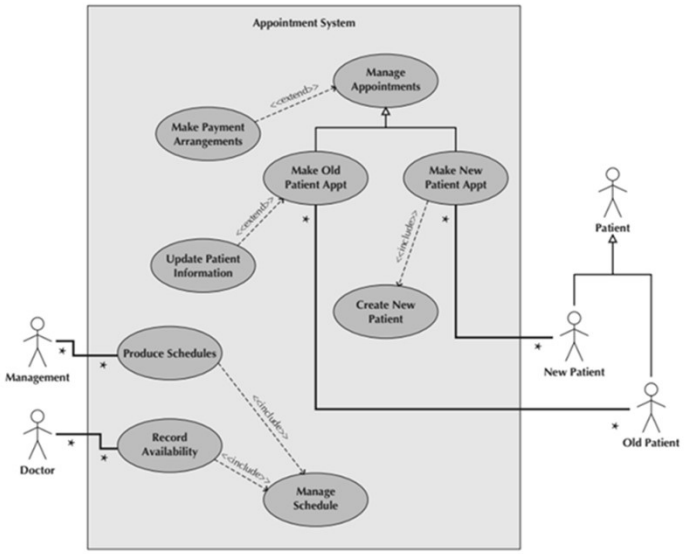
# Use-Case Diagrams

## Elements of Use-Case Diagrams:

| | |
|---|---|
| **An actor:**<br>■ Is a person or system that derives benefit from and is external to the subject.<br>■ Is depicted as either a stick figure (default) or, if a nonhuman actor is involved, a rectangle with <<actor>> in it (alternative).<br>■ Is labeled with its role.<br>■ Can be associated with other actors using a specialization/superclass association, denoted by an arrow with a hollow arrowhead.<br>■ Is placed outside the subject boundary. | <br>Actor/Role<br> <br> |
| **A use case:**<br>■ Represents a major piece of system functionality.<br>■ Can extend another use case.<br>■ Can include another use case.<br>■ Is placed inside the system boundary.<br>■ Is labeled with a descriptive verb–noun phrase. | <br> |
| **A subject boundary:**<br>■ Includes the name of the subject inside or on top.<br>■ Represents the scope of the subject, e.g., a system or an individual business process. | <br> |
| **An association relationship:**<br>■ Links an actor with the use case(s) with which it interacts. | <br> |
| **An include relationship:**<br>■ Represents the inclusion of the functionality of one use case within another.<br>■ Has an arrow drawn from the base use case to the used use case. | <br> |
| **An extend relationship:**<br>■ Represents the extension of the use case to include optional behavior.<br>■ Has an arrow drawn from the extension use case to the base use case. | <br> |
| **A generalization relationship:**<br>■ Represents a specialized use case to a more generalized one.<br>■ Has an arrow drawn from the specialized use case to the base use case. |  |

# Use-Case Diagrams

**Example of Use-Case Diagrams:**




# Course Contents

## Sequence diagrams:

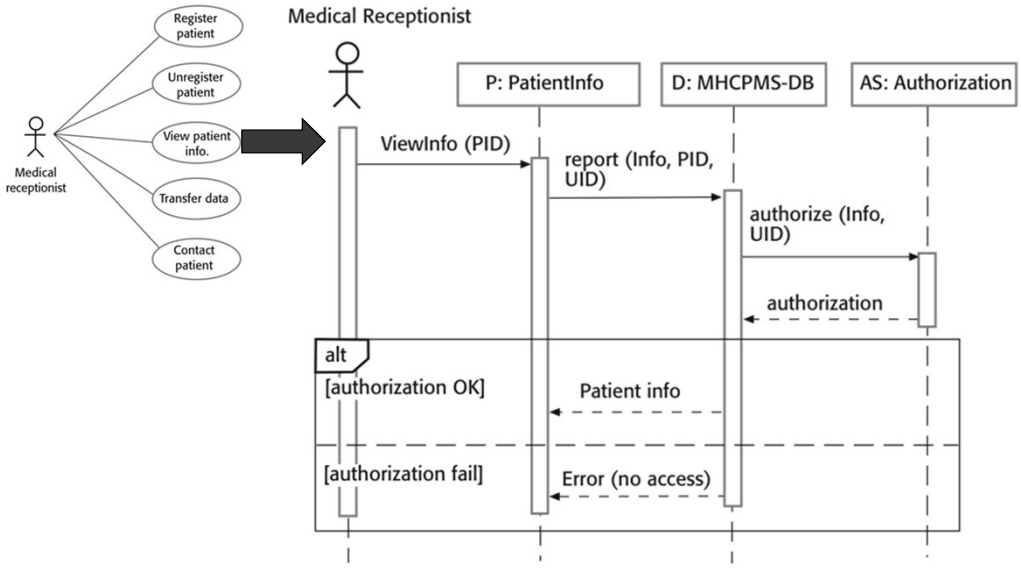
A sequence diagram shows the sequence of interactions that take place during a particular use case or use case instance.

- Sequence diagrams are used to model the interactions between the actors and the objects within a system
- The objects and actors involved are listed along the top of the diagram, with a dotted line drawn vertically from these.
- Interactions between objects are indicated by annotated arrows.

Figure is an example of a sequence diagram that illustrates the basics of the notation. This diagram models the interactions involved in the View patient information use case, where a medical receptionist can see some patient information.

# Course Contents

**Sequence diagram for View patient information:**




# Course Contents

**Sequence diagram for View patient information:**

Figure Can read as follows:

1. The medical receptionist triggers the ViewInfo method in an instance P of the PatientInfo object class, supplying the patient's identifier, PID to identify the required information. P is a user interface object, which is displayed as a form showing patient information.
2. The instance P calls the database to return the information required, supplying the receptionist's identifier to allow security checking. (At this stage, it is not important where the receptionist's UID comes from.)
3. The database checks with an authorization system that the receptionist is authorized for this action.
4. If authorized, the patient information is returned and is displayed on a form on the user's screen. If authorization fails, then an error message is returned.
5. The box denoted by "alt" in the top-left corner is a choice box indicating that one of the contained interactions will be executed. The condition that selects the choice is shown in square brackets.

# Course Contents

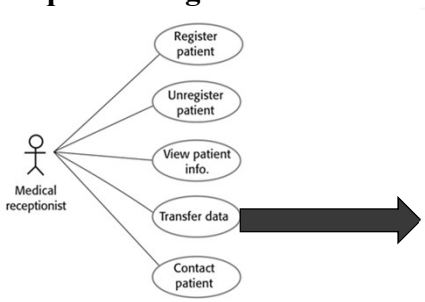
**Sequence diagram for Transfer Data:**

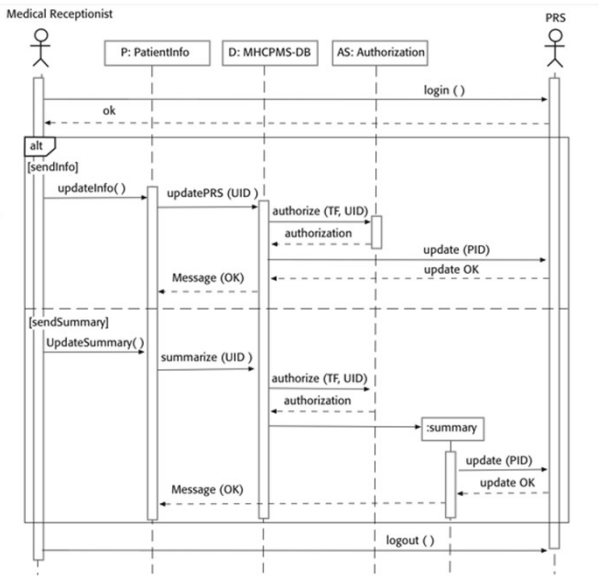
A further example of a sequence diagram from the same system that illustrates two additional features.

- These are the direct communication between the actors in the system and the creation of objects as part of a sequence of operations.
- In this example, an **object of type Summary is created** to hold the summary data that is to be uploaded to a national PRS (Patient Records System).
- To develop system models early in the development process to support requirements engineering and high-level design, there will be many interactions that depend on implementation decisions.

# Course Contents

**Sequence diagram for Transfer Data:**




Example of Sequence Diagram: creation of objects

# Course Contents

**Sequence diagram for Transfer Data:**
The diagram can be read as follows:

1. The receptionist logs on to the PRS.
2. Two options are available (as shown in the “alt” box). These allow the direct transfer of updated patient information from the Mentcare database to the PRS and the transfer of summary health data from the Mentcare database to the PRS.
3. In each case, the receptionist’s permissions are checked using the authorization system.
4. Personal information may be transferred directly from the user interface object to the PRS. Alternatively, a summary record may be created from the database, and that record is then transferred.
5. On completion of the transfer, the PRS issues a status message and the user logs off.

**Sequence Diagram:**

Figure 6-1 shows an instance sequence diagram that depicts the objects and messages for the **Make Old Patient Appt use case**, which describes the process by which an existing patient creates a new appointment or cancels or reschedules an appointment for the doctor’s office appointment system. In this specific instance, the Make Old Patient Appt process is portrayed.

Example of Sequence Diagram: direct communication between the actors

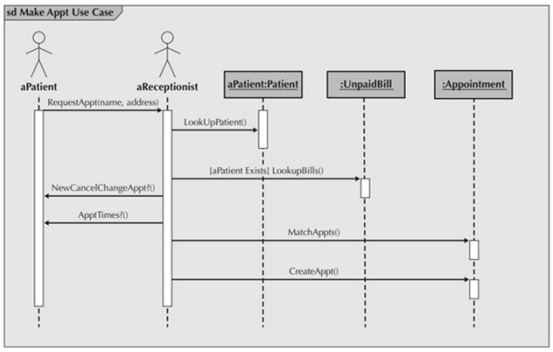



FIGURE 6-1 Example Sequence Diagram

## Elements of a Sequence Diagram:

1. An actor
2. An object
3. A lifeline
4. An execution occurrence
5. A message
6. A guard condition
7. For object destruction
8. A frame

**An actor:**

- Is a person or system that derives benefit from and is external to the system.
- Participates in a sequence by sending and/or receiving messages.
- Is placed across the top of the diagram.
- Is depicted either as a stick figure (default) or, if a nonhuman actor is involved, as a rectangle with <<actor>> in it (alternative).

anActor

<<actor>>
anActor

## Elements of a Sequence Diagram:

**An object:**

- Participates in a sequence by sending and/or receiving messages.
- Is placed across the top of the diagram.

anObject : aClass

**A lifeline:**

- Denotes the life of an object during a sequence.
- Contains an X at the point at which the class no longer interacts.



**An execution occurrence:**

- Is a long narrow rectangle placed atop a lifeline.
- Denotes when an object is sending or receiving messages.

**A message:**

- Conveys information from one object to another one.
- A operation call is labeled with the message being sent and a solid arrow, whereas a return is labeled with the value being returned and shown as a dashed arrow.

aMessage()

ReturnValue

## Elements of a Sequence Diagram:

| | |
|---|---|
| **A guard condition:**<br>■ Represents a test that must be met for the message to be sent. | [aGuardCondition]:aMessage() |
| **For object destruction:**<br>■ An X is placed at the end of an object's lifeline to show that it is going out of existence. | X |
| **A frame:**<br>■ Indicates the context of the sequence diagram. | Context |

# Course Contents

## How to draw Sequence Diagram online:

- One of the most popular solutions to create sequence diagram online is GitMind. The tool is surprisingly simple with lots of amazing features. It comes with free and downloadable templates for easy diagramming. Also, it has an intuitive editor that grants you access to its shapes library for different types of UML diagrams. Moreover, you can export maps to image files or convert them into PDF files.

7 Useful Methods to Create UML Sequence Diagram Online (gitmind.com)

## Course Contents

# Structural models

## Class Diagram

**Review - different models from different perspectives:**

An external perspective, where you model the context or environment of the system.

- An interaction perspective, where you model the interactions between a system and its environment, or between the components of a system.
- A structural perspective, where you model the organization of a system or the structure of the data that is processed by the system.
- A behavioral perspective, where you model the dynamic behavior of the system and how it responds to events.

# Class Diagram

**Structural models:**

Structural models are essential tools to describe, design, and analyze the static aspects of systems, ensuring their components are organized and interconnected effectively.

Structural models are a type of conceptual model that describes the physical and logical organization of a system or entity. They focus on the arrangement and relationships between the components that make up the system, often representing static aspects rather than dynamic behaviors.

There are various types of structural models, including:
Physical Models: These depict the actual physical arrangement of components in a system.

Logical Models: These focus on the logical arrangement and relationships, often used in software engineering to represent classes, objects, and their relationships.

Data Models: These describe how data is structured and related within a database or information system, such as entity-relationship diagrams.

# Class Diagram

**Structural models:**

- **Structural models** of software display the organization of a system in terms of the components that make up that system and their relationships.
- Structural models may be **static** models, which show the structure of the system design.
- WE create structural models of a system when you are discussing and designing the system architecture. These can be models of the overall system architecture or <u>more detailed models of the objects in the system and their relationships</u>.
- Class diagram is use for modeling the static structure of the object classes in a software system.
- Architectural design is an important topic in software engineering, and UML component, package, and deployment diagrams may all be used when presenting architectural models.

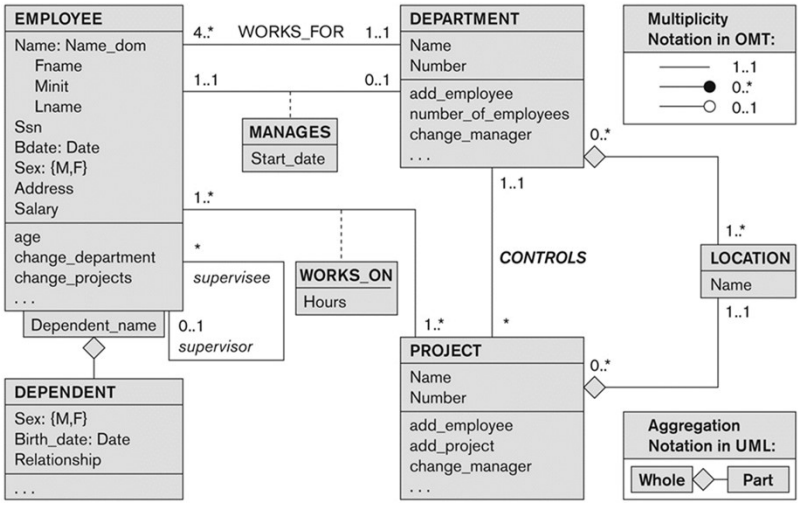
# Class Diagram

**Class diagrams:**

- Class diagrams are used when developing an object-oriented system model to show the classes in a system and the associations between these classes.
- An object class can be thought of as a general definition of one kind of system object.
- An association is a link between classes that indicates that there is some relationship between these classes
- When you are developing models during the early stages of the software engineering process, objects represent something in the real world, such as a patient, a prescription, doctor, etc.

# Course Contents

**Class Diagram:** **Figure 3.16**
The COMPANY conceptual schema in UML class diagram notation.



# Class Diagram

**CLASS DIAGRAMS:**

**Elements of a Class Diagram**

| | |
|---|---|
| **A class:**<br>• Represents a kind of person, place, or thing about which the system will need to capture and store information.<br>• Has a name typed in bold and centered in its top compartment.<br>• Has a list of attributes in its middle compartment.<br>• Has a list of operations in its bottom compartment.<br>• Does not explicitly show operations that are available to all classes. | Class1<br>-Attribute-1<br>+Operation-1() |
| **An attribute:**<br>• Represents properties that describe the state of an object.<br>• Can be derived from other attributes, shown by placing a slash before the attribute's name. | attribute name<br>/derived attribute name |
| **An operation:**<br>• Represents the actions or functions that a class can perform.<br>• Can be classified as a constructor, query, or update operation.<br>• Includes parentheses that may contain parameters or information needed to perform the operation. | operation name () |
| **An association:**<br>• Represents a relationship between multiple classes or a class and itself.<br>• Is labeled using a verb phrase or a role name, whichever better represents the relationship.<br>• Can exist between one or more classes.<br>• Contains multiplicity symbols, which represent the minimum and maximum times a class instance can be associated with the related class instance. | AssociatedWith<br>0..* 1 |
| **A generalization:**<br>• Represents a-kind-of relationship between multiple classes. | |
| **An aggregation:**<br>• Represents a logical a-part-of relationship between multiple classes or a class and itself.<br>• Is a special form of an association. | 0..* IsPartOf ▸ 1 |
| **A composition:**<br>• Represents a physical a-part-of relationship between multiple classes or a class and itself<br>• Is a special form of an association. | 1..* IsPartOf ▸ 1 |

# Class Diagram

**Elements of a Class Diagram: Class, Attributes, Operation**

- Class
- Attributes, calculated or derived attributes
- Visibility: (public (+), protected (#), or private (−)
- Operations: Four kinds of operations that a class can contain: constructor, query, update, and destructor.

# Class Diagram

**CLASS DIAGRAMS:**

**FIGURE Multiplicity**

| | | | |
|---|---|---|---|
| Exactly one | 1 | Department — 1 — Boss | A department has one and only one boss. |
| Zero or more | 0..* | Employee — 0..* — Child | An employee has zero to many children. |
| One or more | 1..* | Boss — 1..* — Employee | A boss is responsible for one or more employees. |
| Zero or one | 0..1 | Employee — 0..1 — Spouse | An employee can be married to zero or one spouse. |
| Specified range | 2..4 | Employee — 2..4 — Vacation | An employee can take from two to four vacations each year. |
| Multiple, disjoint ranges | 1..3,5 | Employee — 1..3,5 — Committee | An employee is a member of one to three or five committees. |

# Course Contents

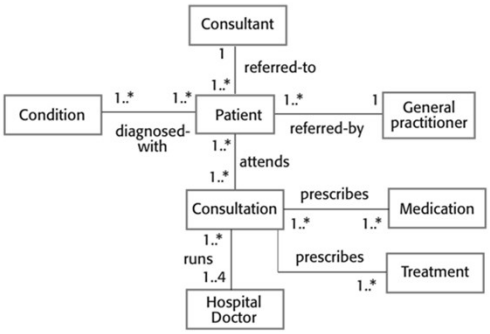
**UML classes and association:**

- Class diagrams in the UML can be expressed at different levels of detail. When you are developing a model, the first stage is usually to look at the world, identify the essential objects, and represent these as classes.
- The simplest way of writing class diagrams is to write the class name in a box and the existence of an association by drawing a line between classes.
- For example, Figure is a simple class diagram showing two classes, Patient and Patient Record, with an association between them.
- Each end of the association is annotated with a 1, meaning that there is a 1:1 relationship between objects of these classes. That is, each patient has exactly one record, and each record maintains information about exactly one patient.

# Course Contents

**Classes and associations in the MHC-PMS:**

Figure show that objects of class Patient are also involved in relationships with a number of other classes. Figure show how many objects are involved in the association.



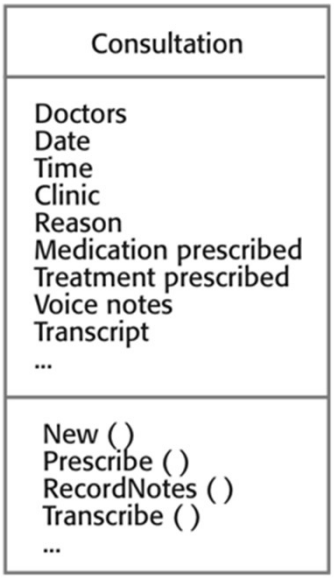

Association can define that an exact number of objects are involved (e.g., 1..4) or, by using a *, indicate that there are an indefinite number of objects involved in the association. For example, the (1..*) multiplicity in Figure on the relationship between Patient and Condition shows that a patient may suffer from several conditions and that the same condition may be associated with several patients.

# Course Contents

**The Consultation class:**

When showing the associations between classes, it is best to represent these classes in the simplest possible way, without attributes or operations.
To define objects in more detail, add information about their attributes (the object's characteristics) and operations (the object's functions).

In the UML, show attributes and operations by extending the simple rectangle that represents a class.

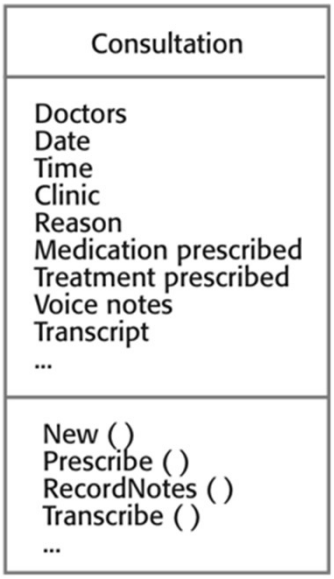


# Course Contents

**The Consultation class:**

Figure shows an object representing a consultation between doctor and patient:

1. The name of the object class is in the top section.
2. The class attributes are in the middle section. This includes the attribute names and, optionally, their types.
3. The operations (called methods in Java and other OO programming languages) associated with the object class are in the lower section of the rectangle.




## Structural Models: Relationships

There are many different types of relationships that can be defined, but all can be classified into three basic categories of data abstraction mechanisms:

1. Generalization relationships
2. Aggregation relationships
3. Association relationships

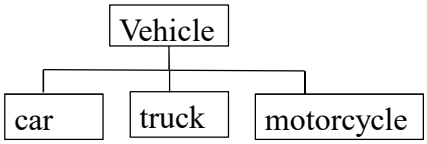
These data-abstraction mechanisms allow the analyst to focus on the important dimensions while ignoring non-essential dimensions. Like attributes, the analyst must be careful to include only relevant relationships.

## **Relationships -** Generalization Relationships

Generalization represents the "is-a" relationship or inheritance, where one class (the subclass or derived class) inherits properties and behaviors from another class (the superclass or base class). The subclass is a specialized version of the superclass. Generalization allows us to model hierarchical relationships and create a class hierarchy.

For example, consider a class hierarchy for vehicles: "Vehicle" is the superclass, and "Car," "Truck," and "Motorcycle" are subclasses that inherit from the "Vehicle" class.

For example, a customer class and an employee class can be generalized into a person class by extracting the attributes and operations they have in common and placing them into the new superclass, person. In this way, the analyst can reduce the redundancy in the class definitions so that the common elements are defined once and then reused in the subclasses. Generalization is represented with the a-kind-of relationship, so that we say that an employee is a-kind-of person.




## **Relationships -** Generalization Relationships

The analyst also can use the flip side of generalization, specialization, to uncover additional classes by allowing new subclasses to be created from an existing class. For example, an employee class can be specialized into a secretary class and an engineer class. Furthermore, generalization relationships between classes can be combined to form generalization hierarchies. Based on the previous examples, a secretary class and an engineer class can be subclasses of an employee class, which in turn could be a subclass of a person class.

This would be read as a secretary and an engineer are a-kind-of employee and a customer and an employee are a-kind-of person.

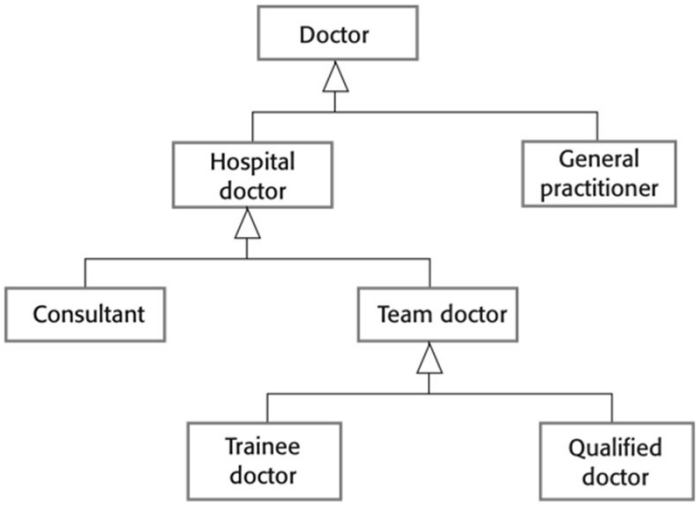
# Course Contents

**Generalization:**

- In modeling systems, it is often useful to examine the classes in a system to see if there is scope for generalization. If changes are proposed, then you do not have to look at all classes in the system to see if they are affected by the change.
- In object-oriented languages, such as Java, generalization is implemented using the class inheritance mechanisms built into the language
- In a generalization, the attributes and operations associated with higher-level classes are also associated with the lower-level classes
- The lower-level classes are subclasses inherit the attributes and operations from their superclasses. These lower-level classes then add more specific attributes and operations.

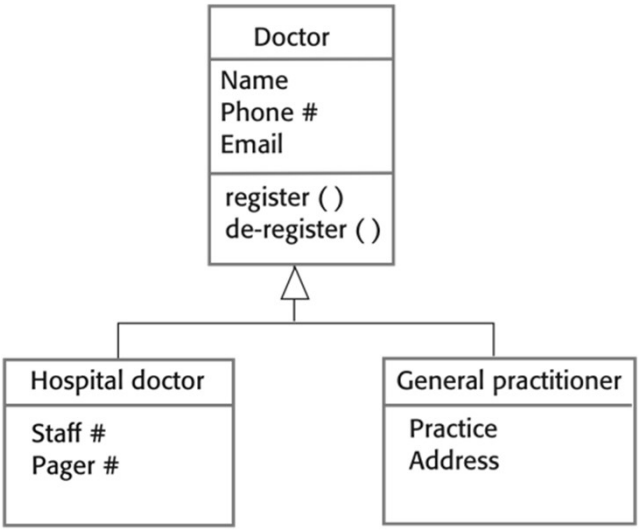
# Course Contents

**A generalization hierarchy:**



# Course Contents

**A generalization hierarchy with added detail:**




# Course Contents

**Generalization:**

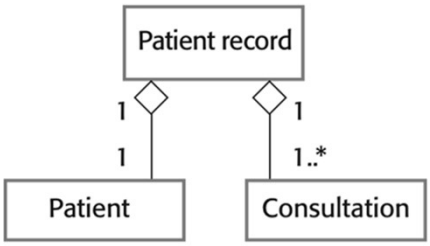
- Generalization is an everyday technique that we use to manage complexity.
- Rather than learn the detailed characteristics of every entity that we experience, we place these entities in more general classes (animals, cars, houses, etc.) and learn the characteristics of these classes.
- This allows us to infer that different members of these classes have some common characteristics.

## Relationships - Aggregation Relationships

Aggregation represents a **"has-a" or "contains"** relationship between classes. It models a whole-part relationship, where one class represents a whole object that is composed of other objects (parts). Aggregation allows objects to be connected in a more loosely coupled manner.

For instance, consider a class for "University" that aggregates "Department" objects. The University contains multiple departments, but the existence of departments is independent of the university itself.

Patient record contains **whole part** of Patient class and **part of** Consultation class objects.




## Relationships - Aggregation Relationships

**Aggregation** or **composition** is a relationship among classes by which a class can be made up of any combination of objects of other classes. It allows objects to be placed directly within the body of other classes. Aggregation is referred as a “part–of” or “has–a” relationship, with the ability to navigate from the whole to its parts. An aggregate object is an object that is composed of one or more other objects.

Example:

In the relationship, “a car has–a motor”, car is the whole object or the aggregate, and the motor is a “part–of” the car. Aggregation may denote −

Physical containment − Example, a computer is composed of monitor, CPU, mouse, keyboard, and so on.

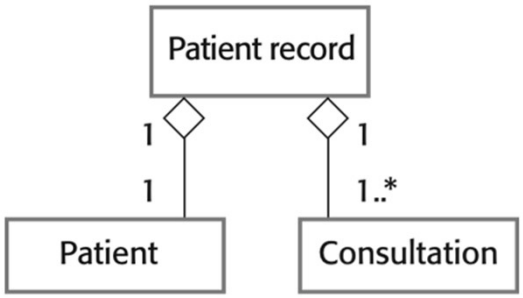
Conceptual containment − Example, shareholder has–a share.

# Course Contents

**Object class aggregation models:**

- An aggregation model shows how classes that are collections, are composed of other classes
- Aggregation models are similar to the part-of relationship in semantic data models

**The aggregation association:**




## Relationships - **Association Relationships**

Association represents a relationship between classes where objects of one class are associated with objects of another class. It shows how different classes collaborate or interact with each other. Associations can be one-to-one, one-to-many, or many-to-many.
For example, consider a class for "Student" and a class for "Course." There can be an association between them representing that each student can take multiple courses, and each course can be taken by multiple students.

When creating class relationships, it's essential for the analyst to carefully include only relevant relationships to avoid unnecessary complexity and ensure a clear and concise representation of the domain model. This helps in focusing on the essential dimensions of the problem while ignoring nonessential details, making the model more manageable and maintainable.

## Course Contents

**Behavioral models:**

- Behavioral models are models of the dynamic behavior of a system as it is executing. They show what happens or what is supposed to happen when a system responds to a stimulus (provocation) from its environment.
- It's all about things happening in response to events, like when one thing sets off another, just like how a chain reaction work.

You can think of these stimuli as being of two types:

- Data: Data becomes available that has to be processed by the system. The availability of the data triggers the processing.
- Events: Some event happens that triggers system processing. Events may have associated data, although this is not always the case.

**Example-** Online Marketplace: Notifying users about product discounts onnew arrivals. IoT Devices: Reacting to sensor data to automate processes or trigger alert.

## Course Contents

# Behavioral models

## Course Contents

**Behavioral models:**

Behavioral models are used to depict the dynamic behavior of a system. These models focus on illustrating how the system behaves and interacts with its users, other systems, and the environment over time.

- Many business systems are data-processing systems that are primarily driven by data. They are controlled by the data input to the system, with relatively little external event processing. Their processing involves a sequence of actions on that data and the generation of an output.

For example, a phone billing system will accept information about calls made by a customer, calculate the costs of these calls, and generate a bill for that customer.

- Based on the stimuli types, Behavioral models may be
  1. Data-driven modeling
  2. Event-driven modeling

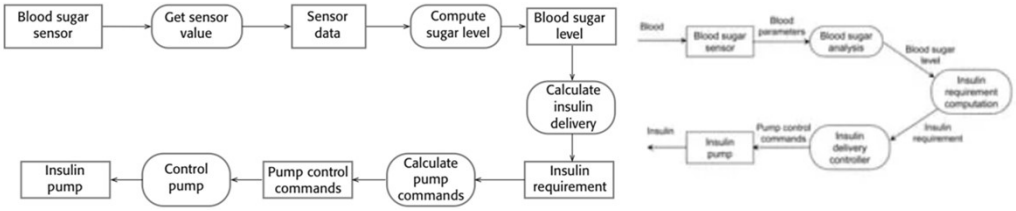
## Course Contents

**Data-driven modeling:**

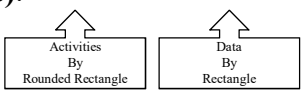
- Data-driven models show the sequence of actions involved in processing input data and generating an associated output.
- They are particularly useful during the analysis of requirements as they can be used to show end-to-end processing in a system.
- In the 1970s, structured design methods used data-flow diagrams (DFDs) as a way to illustrate the processing steps in a system.
- Data-flow diagrams can be represented in the UML using the activity diagram type. An alternative way of showing the sequence of processing in a system is to use UML sequence diagrams. So, Data-driven models can be created using UML activity diagrams and UML sequence diagrams.

# Course Contents

**Data-driven modeling: An activity model of the insulin pump's operation**

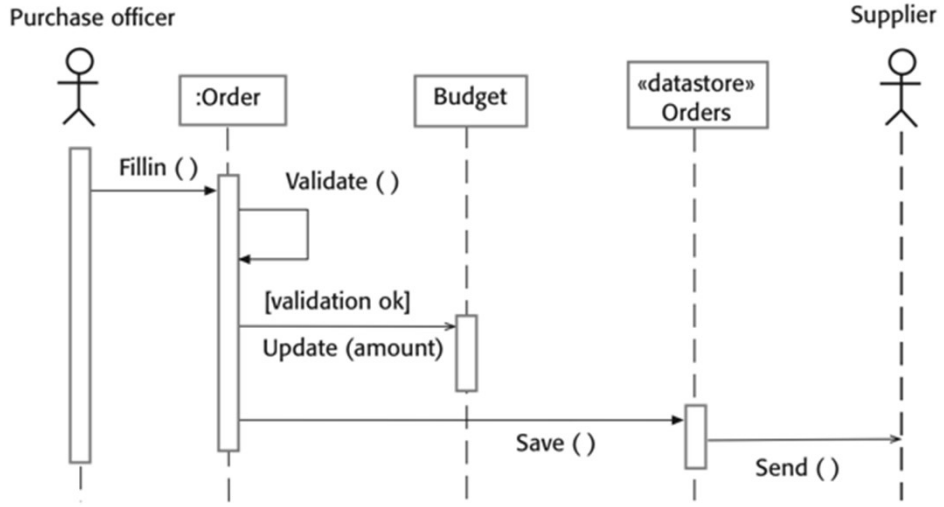



As an example shown above, a simple activity diagram that shows the chain of processing involved in the insulin pump software. The processing steps, represented as **activities (rounded rectangles)**, and the data flowing between these steps, represented as **objects (rectangles)**.



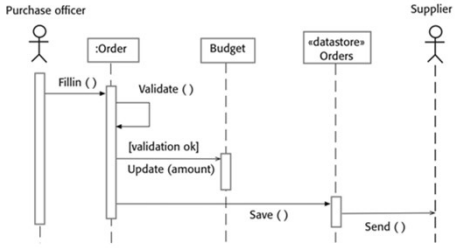

# Course Contents

**Data-driven modeling: Sequence model of processing an order and sending it to a supplier**



# Course Contents

**Data-driven modeling: Sequence model of processing an order and sending it to a supplier**




Sequence models highlight objects in a system, whereas data-flow diagrams highlight the operations or activities.
In practice, **non-experts** seem to find data-flow diagrams more intuitive, but **engineers** prefer sequence diagrams.

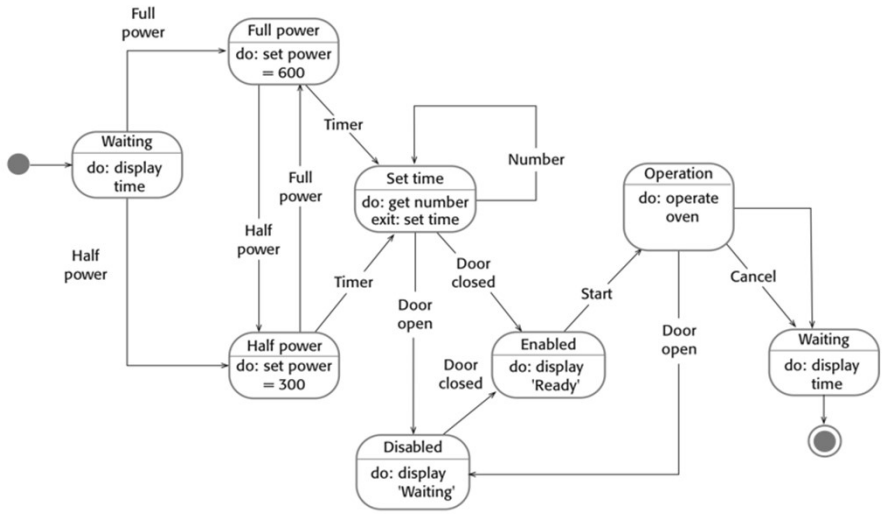
# Course Contents

**Event-driven modeling:**

- Real-time systems are often event-driven, with minimal data processing. For example, a landline phone switching system responds to events such as ‘receiver off hook’ by generating a dial tone.
- Event-driven modeling shows how a system responds to external and internal events.
- It is based on the assumption that a system has a finite number of states and that events (stimuli) may cause a transition from one state to another.
- Event-driven modeling is used extensively when designing and documenting real-time systems.
- The UML supports event-based modeling using state diagrams, which are based on Statecharts.

# Course Contents

**Event-driven modeling: State diagram of control software for a microwave oven to illustrate event-driven modeling**




# Course Contents

**Event-driven modeling: State diagram of control software for a microwave oven to illustrate event-driven modeling**

This simple oven has a switch to select **full or half power**, a **numeric keypad to input the cooking time**, a **start/stop button**, and an alphanumeric display.

The sequence of actions in using the microwave is as follows:

1. Select the power level (either half power or full power).
2. Input the cooking time using a numeric keypad.
3. Press Start and the food is cooked for the given time.

In UML state diagrams, rounded rectangles represent system states. They may include a brief description (following "do") of the actions taken in that state. The labeled arrows represent stimuli that force a transition from one state to another. Start and end states can indicate using filled circles, as in activity diagrams.

# Course Contents

**Event-driven modeling: State diagram of control software for a microwave oven to illustrate event-driven modeling**

The system starts in a waiting state and responds initially to either the full-power or the half-power button. Users can change their minds after selecting one of these and may press the other button. The time is set and, if the door is closed, the Start button is enabled. Pushing this button starts the oven operation, and cooking takes place for the specified time. This is the end of the cooking cycle, and the system returns to the waiting state.

State models of a system provide an overview of event processing, but you normally have to extend this with a more detailed description of the stimuli and the system states. We may use a table to list the states and events that stimulate state transitions along with a description of each state and event. Table below shows a tabular description of each state and how the stimuli that force state transitions are generated.

# Course Contents

**States and stimuli for the microwave oven (a):**

Table below shows a tabular description of each state and how the stimuli that force state transitions are generated.

| State | Description |
|---|---|
| Waiting | The oven is waiting for input. The display shows the current time. |
| Half power | The oven power is set to 300 watts. The display shows 'Half power'. |
| Full power | The oven power is set to 600 watts. The display shows 'Full power'. |
| Set time | The cooking time is set to the user's input value. The display shows the cooking time selected and is updated as the time is set. |
| Disabled | Oven operation is disabled for safety. Interior oven light is on. Display shows 'Not ready'. |
| Enabled | Oven operation is enabled. Interior oven light is off. Display shows 'Ready to cook'. |
| Operation | Oven in operation. Interior oven light is on. Display shows the timer countdown. On completion of cooking, the buzzer is sounded for five seconds. Oven light is on. Display shows 'Cooking complete' while buzzer is sounding. |

# Course Contents

**States and stimuli for the microwave oven (b):**

| Stimulus | Description |
|---|---|
| Half power | The user has pressed the half-power button. |
| Full power | The user has pressed the full-power button. |
| Timer | The user has pressed one of the timer buttons. |
| Number | The user has pressed a numeric key. |
| Door open | The oven door switch is not closed. |
| Door closed | The oven door switch is closed. |
| Start | The user has pressed the Start button. |
| Cancel | The user has pressed the Cancel button. |

# Course Contents

**State machine models:**

- These model the behaviour of the system in response to external and internal events.
- They show the system's responses to stimuli so are often used for modelling real-time systems.
- State machine models show system states as nodes and events as arcs between these nodes. When an event occurs, the system moves from one state to another.
- Statecharts are an integral part of the UML and are used to represent state machine models.

**Elements of a Behavioral State Machine:**

Figure below presents an example of a behavioral state machine representing the patient class in the context of a hospital environment. From this diagram, we can tell that a patient **enters** a hospital and is **admitted** after checking in. If a doctor finds the patient to be healthy, he or she is **released** and is no longer considered a patient after two weeks elapse. If a patient is found to be unhealthy, he or she remains **under observation** until the diagnosis changes.

Six elements:

1. A state
2. An initial state
3. A final state
4. An event
5. A transition
6. A frame

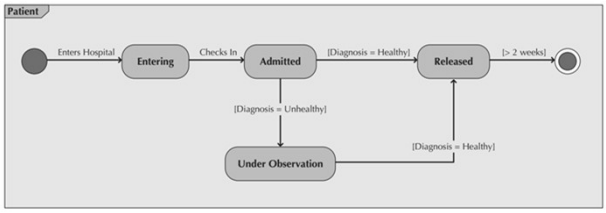



FIGURE 6-16 Sample Behavioral State Machine Diagram

**Elements of a Behavioral State Machine:**

Six elements:

1. A state
2. An initial state
3. A final state
4. An event
5. A transition
6. A frame

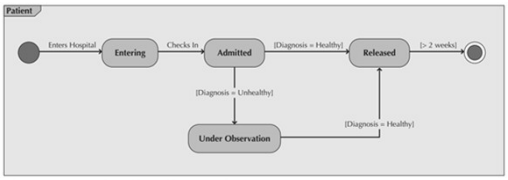



FIGURE 6-16 Sample Behavioral State Machine Diagram

| | |
|---|---|
| **A state:**<br>■ Is shown as a rectangle with rounded corners.<br>■ Has a name that represents the state of an object. | aState |
| **An initial state:**<br>■ Is shown as a small, filled-in circle.<br>■ Represents the point at which an object begins to exist. | |
| **A final state:**<br>■ Is shown as a circle surrounding a small, filled-in circle (bull's-eye).<br>■ Represents the completion of activity. | |

## Elements of a Behavioral State Machine:

Six elements:

1. A state
2. An initial state
3. A final state
4. An event
5. A transition
6. A frame

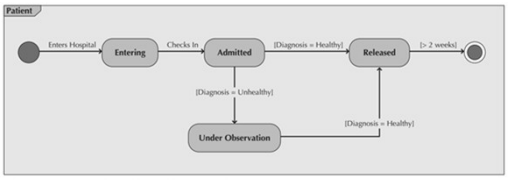



**FIGURE 6-16** Sample Behavioral State Machine Diagram

| | |
|---|---|
| **An event:**<br>■ Is a noteworthy occurrence that triggers a change in state.<br>■ Can be a designated condition becoming true, the receipt of an explicit signal from one object to another, or the passage of a designated period of time.<br>■ Is used to label a transition. | anEvent |
| **A transition:**<br>■ Indicates that an object in the first state will enter the second state.<br>■ Is triggered by the occurrence of the event labeling the transition.<br>■ Is shown as a solid arrow from one state to another, labeled by the event name. | → |
| **A frame:**<br>■ Indicates the context of the behavioral state machine. | Context |

# Course Contents

**Model-driven engineering:**

- Model-driven engineering (MDE) is an approach to software development where models rather than programs are the principal outputs of the development process.
- The programs that execute on a hardware/software platform are then generated automatically from the models.
- Proponents (supporters) of MDE argue that this raises the level of abstraction in software engineering so that engineers no longer have to be concerned with programming language details or the specifics of execution platforms.

## Course Contents

**Usage of model-driven engineering:**

- Model-driven engineering is still at an early stage of development, and it is unclear whether or not it will have a significant effect on software engineering practice.
- Pros
  - Allows systems to be considered at higher levels of abstraction
  - Generating code automatically means that it is cheaper to adapt systems to new platforms
- Cons
  - Models for abstraction are not necessarily right for implementation
  - Savings from generating code may be outweighed(even exceeded) by the costs of developing translators for new platforms

## Course Contents

**Model driven architecture:** Model-Driven Architecture

Model-Driven Architecture (MDA) is a framework for software development processes that is at the core of the Object Management Group's (OMG) recommendations:

**There are three models in MDA:**
CIM = Computation-Independent Model
PIM = Platform-Independent Model
PSM = Platform-Specific Model

All three models may consist of one or more UML diagrams.

- The CIM combines the requirements and domain models. It models the system in terms of how it will interact with its environment.
- The PIM is the design model. It describes the internal structure of the model without regard to the hosting platform.
- The PSM is the implementation model. It adds concepts from the hosting platform to the hosting platform. Examples of platforms include .Net, J2SE, J2ME, J2EE, CORBA.

## Course Contents

**Model driven architecture:**

- Model-driven architecture (MDA) was the precursor(ancestor) of the more general model-driven engineering
- MDA is a model-focused approach to software design and implementation that uses a subset of UML models to describe a system.
- Models at different levels of abstraction are created. From a high-level, platform independent model, it is possible, in principle, to generate a working program without manual intervention.

## Course Contents

**Types of model:** The MDA method recommends that three types of abstract system model should be produced:

- **A computation independent model (CIM)**

A software or system model which defines functionality in a declarative manner, independent of specific algorithms to carry out the functionality. These model the important domain abstractions used in a system. CIMs are sometimes called domain models.

- **A platform independent model (PIM)**

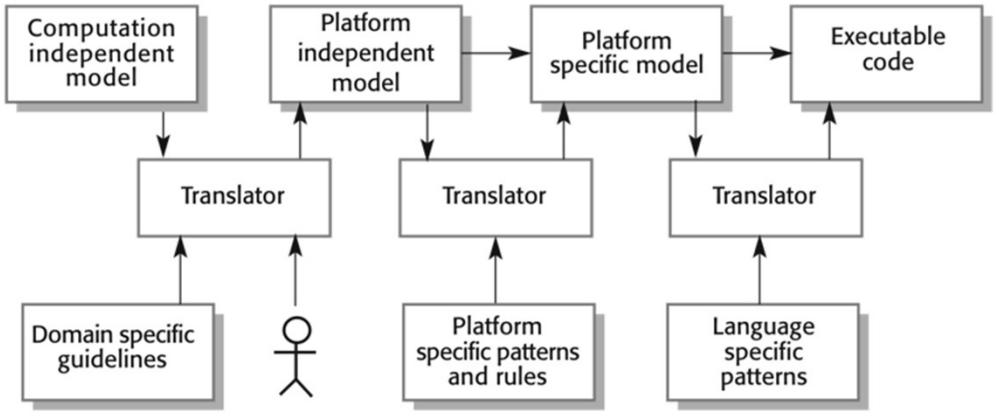
These model the operation of the system without reference to its implementation. The PIM is usually described using UML models that show the static system structure and how it responds to external and internal events.

- **Platform specific models (PSM)**

These are transformations of the platform-independent model with a separate PSM for each application platform. In principle, there may be layers of PSM, with each layer adding some platform-specific detail.

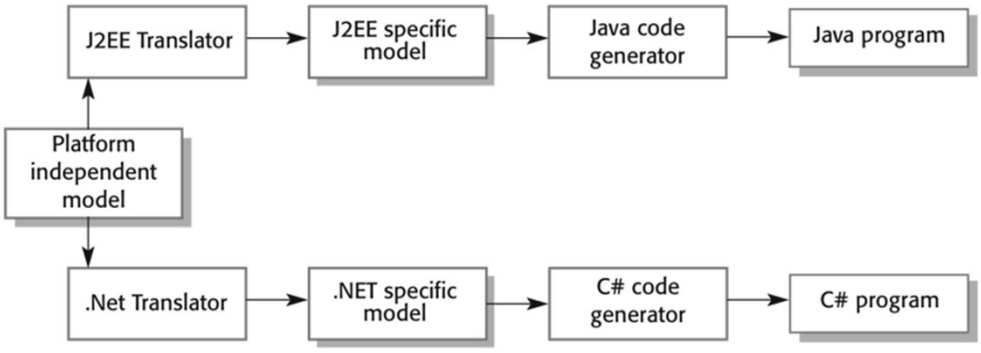
# Course Contents

**MDA transformations:**




# Course Contents

**Multiple platform-specific models:**



## Course Contents

**Key points:**

- A model is an abstract view of a system that ignores system details. Complementary system models can be developed to show the system's context, interactions, structure and behavior.
- Context models show how a system that is being modeled is positioned in an environment with other systems and processes.
- Use case diagrams and sequence diagrams are used to describe the interactions between users and systems in the system being designed. Use cases describe interactions between a system and external actors; sequence diagrams add more information to these by showing interactions between system objects.
- Structural models show the organization and architecture of a system. Class diagrams are used to define the static structure of classes in a system and their associations.

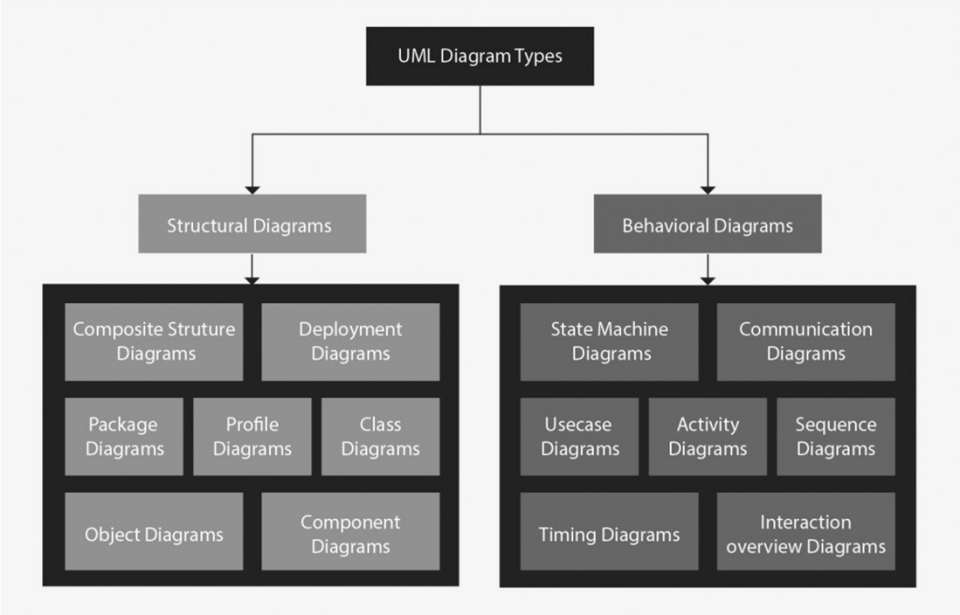
## Course Contents

**Key points:**

- Behavioral models are used to describe the dynamic behavior of an executing system. This behavior can be modeled from the perspective of the data processed by the system, or by the events that stimulate responses from a system.
- Activity diagrams may be used to model the processing of data, where each activity represents one process step.
- State diagrams are used to model a system's behavior in response to internal or external events.
- Model-driven engineering is an approach to software development in which a system is represented as a set of models that can be automatically transformed to executable code.

# Reference Slides

## What is a software process model?



**Communication Diagrams:**

Communication diagrams are equivalent to sequence diagrams, but they emphasize the flow of messages through a set of objects, whereas the sequence diagrams focus on the time ordering of the messages being passed.

Therefore, to understand the flow of control over a set of collaborating objects or to understand which objects collaborate to support business processes, a communication diagram can be used. For time ordering of the messages, a sequence diagram should be used. In some cases, both can be used to more fully understand the dynamic activity of the system.

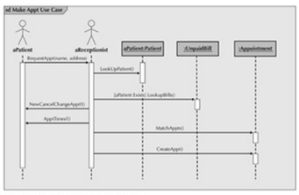



FIGURE 6-1 Example Sequence Diagram

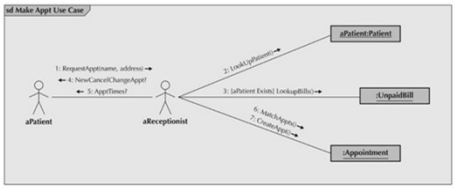



FIGURE 6-10 Sample Communication Diagram

**Communication Diagrams:**

- Communication diagrams helps to depict how objects in a system interact through message passing.
- Communication diagrams, like sequence diagrams, essentially provide a view of the dynamic aspects of an object-oriented system. They can show how the members of a set of objects collaborate to implement a use case or a use-case scenario.
- They can also be used to model all the interactions among a set of collaborating objects, in other words, a collaboration (CRC cards). In this case, a communication diagram can portray how dependent the different objects are on one another.
- A communication diagram is essentially an object diagram that shows message-passing relationships instead of aggregation or generalization associations. Communication diagrams are very useful to show process patterns (i.e., patterns of activity that occur over a set of collaborating classes).

**Communication Diagrams:**

Elements of a Communication Diagram shows a communication diagram for the Make Old Patient Appt use case. Like the sequence diagram, the Make Old Patient Appt process is portrayed.

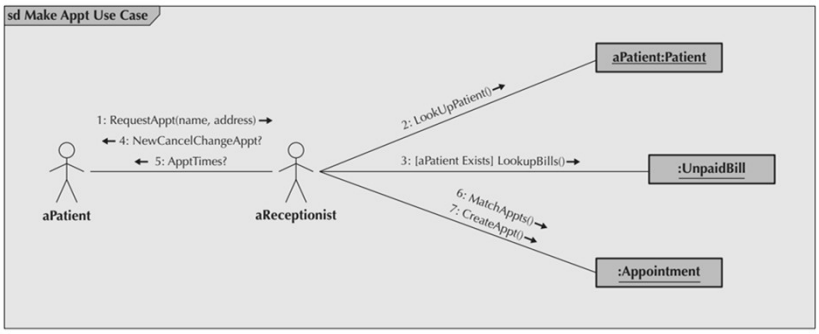



**FIGURE 6-10** Sample Communication Diagram

**Elements of a Communication Diagram:** **Communication Diagram Syntax**

1. An actor:
2. An object
3. An association:
4. A message:
5. A guard condition:
6. A frame:

**An actor:**

- Is a person or system that derives benefit from and is external to the system.
- Participates in a collaboration by sending and/or receiving messages.
- Is depicted either as a stick figure (default) or, if a nonhuman actor is involved, as a rectangle with <<actor>> in it (alternative).

anActor

<<actor>>
anActor

**An object:**

- Participates in a collaboration by sending and/or receiving messages.

anObject : aClass

## Elements of a Communication Diagram: Communication Diagram Syntax

| | |
|---|---|
| **An association:**<br>■ Shows an association between actors and/or objects.<br>■ Is used to send messages. | |
| **A message:**<br>■ Conveys information from one object to another one.<br>■ Has direction shown using an arrowhead.<br>■ Has sequence shown by a sequence number. | SeqNumber: aMessage → |
| **A guard condition:**<br>■ Represents a test that must be met for the message to be sent. | SeqNumber: [aGuardCondition]: aMessage → |
| **A frame:**<br>■ Indicates the context of the communication diagram. | Context |